ML THEORY ASSIGNMENT · PROJECT REPORT

K-Nearest Neighbours for Breast Cancer Diagnosis

Implementation, optimisation and overfitting analysis on the Wisconsin Diagnostic Breast Cancer dataset

Swayam Mallick, Krishnendu Dhar, Rwitama Aon, Pragati Kumari
Techno India University, West Bengal · Batch BCS 3B · November 2026

Code: github.com/Swayam424/breast-cancer-knn-optimization

96.55%	97.37%	95.24%	5.3 pts
K-NN (k=7) accuracy, repeated 5-fold CV (± 1.61)	K-NN (k=7) accuracy on the held-out test split	Malignant recall with the 3-of-7 rule (2 of 42 missed)	Train-validation gap at k=1, the clearest overfitting signal

1. Summary

This project applies the K-Nearest Neighbours (K-NN) classifier to a real clinical task: deciding whether a breast tumour is benign or malignant from measurements of its cell nuclei. Scope follows the updated assignment requirement, which specifies K-NN. A standard K-NN with k=7 reaches 97.37% on the test split, and repeated cross-validation puts its expected accuracy at 96.55% ± 1.61, the more reliable figure.

The optimisation work shows that this dataset leaves little accuracy to gain: most tuned variants fall within the noise of the baseline. The useful gain is clinical. Lowering the malignant vote threshold from 4 of 7 to 3 of 7 catches one more cancer, at the cost of 6 false alarms. A differentiable Soft-KNN raises recall further but overfits quickly: its training loss reaches almost zero while validation loss climbs.

2. Real-life problem and purpose

Fine-needle aspiration is a common way to examine a suspicious breast lump. The two possible errors are not equally serious: a false alarm leads to further tests, while a missed malignancy delays treatment. The project therefore tracks **malignant recall** alongside accuracy, and asks three questions:

- How accurately can K-NN separate malignant from benign tumours?
- Which optimisation techniques actually help, once random variation is accounted for?
- Where does the model overfit or underfit, and which settings avoid both?

This is a coursework study on a small benchmark. It is not a validated medical device and must not guide patient care.

3. Dataset

The Wisconsin Diagnostic Breast Cancer (WDBC) dataset holds 569 tumour samples, 357 benign and 212 malignant, each described by 30 continuous features such as radius, texture, concavity and smoothness. The dataset originates from the UCI Machine Learning Repository and is also published on Kaggle as *uciml/breast-cancer-wisconsin-data*. The code loads the identical data through scikit-learn, so no download is needed.

Data is split 80/20 with stratification (seed 42), giving 455 training and 114 test samples, 42 of them malignant. Features range across several orders of magnitude, so they are standardised using statistics from the training data only.

4. Method

For a query sample, K-NN finds the *k* closest training samples and predicts the majority class among them. It has no training phase, so its behaviour is set by the choice of *k*, the distance metric, the feature space and the decision threshold. The pipeline runs in six steps:

- **Load and split** the data, with the test set held out.
- **Standardise** using training statistics only, refitting inside every cross-validation fold.
- **Benchmark distance metrics:** Euclidean, Manhattan, cosine and Mahalanobis.
- **Tune:** sweep *k* from 1 to 25 and compress features with PCA.
- **Optimise:** train a differentiable Soft-KNN, and compare optimisers, activations and augmentation.
- **Calibrate and evaluate:** set the vote threshold, then report accuracy, recall, macro F1 and error counts.

Soft-KNN. Hard voting has no useful gradient, so it cannot be trained by backpropagation. Soft-KNN passes inputs through a small neural network (64 hidden units, 32-dimensional normalised output), then applies a softmax with temperature 0.1 to the negative mean distance to the 5 nearest samples of each class. The network can then learn a distance space using cross-entropy loss.

5. Baseline results

Configuration	Test accuracy	Malignant recall	Macro F1	Missed cancers	False alarms
Euclidean, k=5	95.61%	92.86%	0.9526	3	2
Manhattan, k=5	96.49%	92.86%	0.9619	3	1
Cosine, k=5	95.61%	92.86%	0.9526	3	2
Mahalanobis, k=5	81.58%	50.00%	0.7697	21	0
Euclidean, k=7	97.37%	92.86%	0.9713	3	0
PCA d=2, k=7	92.11%	90.48%	0.9156	4	5
PCA d=6, k=7	96.49%	95.24%	0.9623	2	2
PCA d=10, k=7	95.61%	92.86%	0.9526	3	2
PCA d=15, k=7	95.61%	92.86%	0.9526	3	2

Table 1. K-NN on the held-out test split (114 samples). PCA retains 63.4%, 89.1%, 95.3% and 98.7% of variance at d = 2, 6, 10 and 15.

Euclidean, Manhattan and cosine distances perform alike. Mahalanobis is the outlier, missing 21 of 42 cancers; a likely cause is that it fully whitens highly correlated features, which inflates low-variance noise directions. With a test split this small, one sample moves accuracy by 0.88 points, and the 95% margin around 97% is about ±3.1 points. Differences of a point or two in Table 1 should not be read as real.

6. Optimisation

The assignment asks how the model can be optimised through optimisers, activation functions, data augmentation and fine-tuning. Each neural result below is averaged over 10 random seeds, because a single seed can be misleading: Soft-KNN with AdamW spans 93.86% to 96.49% depending on the seed.

Technique	Setting	Test accuracy	Malignant recall
Optimiser	AdamW	95.53% ± 0.92	96.43%
	SGD + momentum	95.18% ± 1.13	95.24%
	SGD, no momentum	94.04% ± 1.75	93.57%
Activation	ReLU (AdamW)	95.18% ± 0.71	95.71%
	GELU (AdamW)	95.53% ± 0.92	96.43%
Regularisation	Early stopping at epoch 9	95.26% ± 1.05	95.00%
Ensembling	3 seeds, averaged	95.61%	97.62%
Augmentation	Gaussian jitter, std 0.1 (K-NN, k=7)	97.37% ± 0.78	94.05%
	Gaussian jitter, std 0.3	96.23% ± 0.79	92.86%
	Gaussian jitter, std 0.5	97.19% ± 0.35	92.86%

Table 2. Optimisation experiments. Values show mean ± standard deviation over 10 seeds; the ensemble is one fixed combination of seeds 42, 7 and 123.

- **Optimisers.** AdamW and momentum outperform plain SGD, which also varies most between seeds. Plain SGD has not converged in 60 epochs at this learning rate.
- **Activation.** ReLU and GELU are indistinguishable given the seed-to-seed spread.
- **Augmentation.** Jitter neither clearly helps nor seriously harms K-NN. Adding copies of the training data does not add new information about the tumours.
- **Fine-tuning.** Cross-validation accuracy is nearly flat for k between 3 and 13, so any value there is defensible. Choosing k=7 is reasonable; the single best fold score at k=10 is within the noise.
- **Code speed.** Replacing Python loops with NumPy broadcasting returned identical predictions and ran about 22 times faster (477 ms to 22 ms, median of 5 runs). The ratio depends on the machine.

7. Loss and accuracy graphs

K-NN has no epochs, so its curves are plotted against *k*, using zero-one loss (1 – accuracy). Soft-KNN is trained iteratively, so its curves are plotted against epochs, on a validation split carved from the training data so that the test set stays untouched.

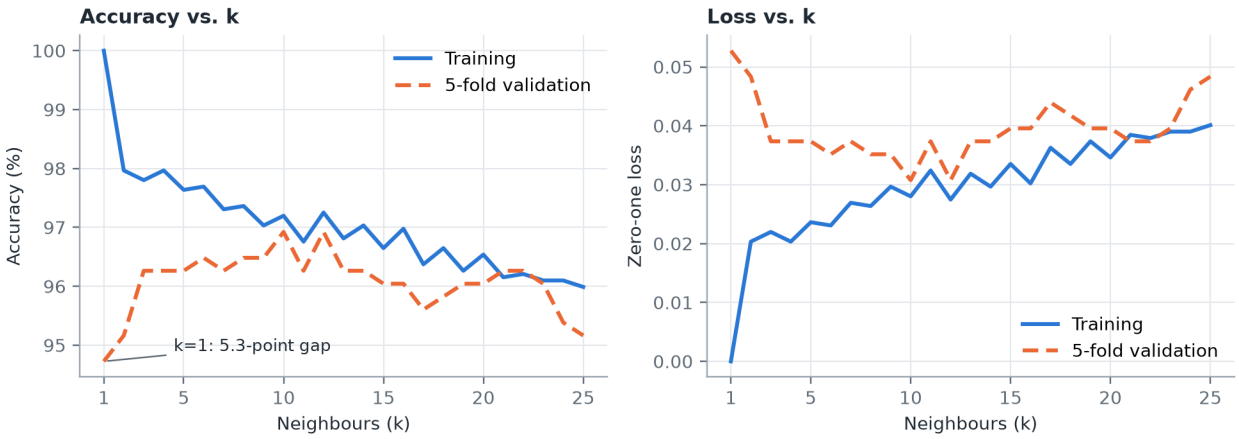


Figure 1. K-NN training and 5-fold validation performance across k. The scaler is refit within each fold.

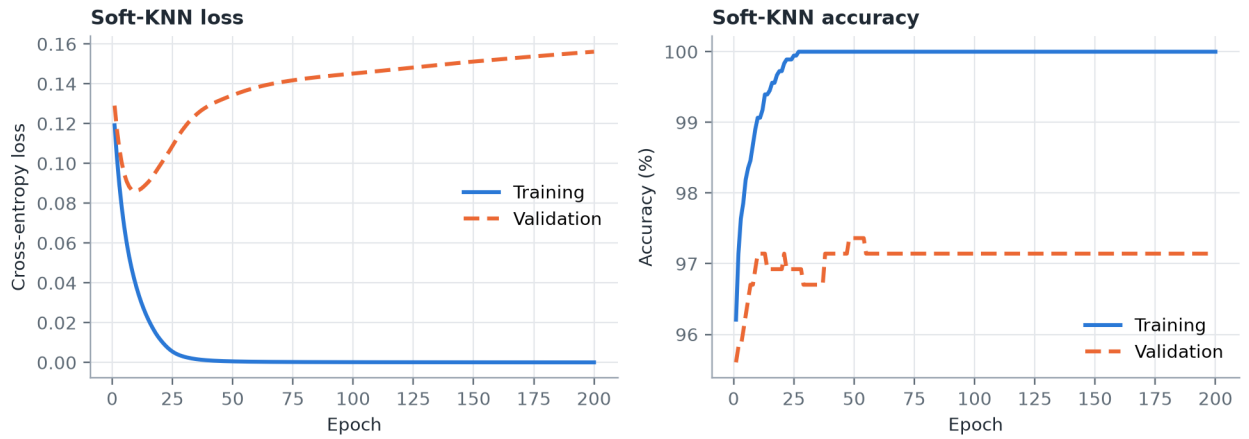


Figure 2. Soft-KNN with AdamW, averaged over 5 seeds, for 200 epochs.

8. Overfitting and underfitting

Overfitting at k=1. Each training sample is its own nearest neighbour, so training accuracy is a perfect 100%, while validation accuracy is 94.73%. This 5.27-point gap is the signature of high variance: the model memorises noise.

Balanced at k=7. Training (97.31%) and validation (96.26%) sit close together with a 1.04-point gap, while validation accuracy is near its plateau.

Mild underfitting at k=25. Training accuracy falls to 95.99% and validation to 95.16%. The gap is small (0.82 points) but both scores are lower, which points to rising bias. The effect is mild because the two classes are well separated. Aggressive PCA gives a similar picture: at d=2, test accuracy drops to 92.11%.

Overfitting in Soft-KNN. Validation loss reaches its minimum of 0.086 at epoch 9 and then rises to 0.156 by epoch 200, while training loss falls to almost zero and training accuracy reaches 100%. Validation accuracy holds at 97.14%: the network keeps classifying well but becomes overconfident, which loss detects and accuracy does not. A model with thousands of parameters is memorising 455 samples. Stopping early at epoch 9 did not raise test accuracy (95.26% ± 1.05) and gave lower recall, so its benefit here is lower validation loss, not more correct answers.

9. Clinical threshold calibration

Lowering the number of malignant votes needed to raise an alert trades false alarms for fewer missed cancers.

Rule	Test accuracy	Malignant recall	Macro F1	Missed cancers	False alarms
4/7 votes	97.37%	92.86%	0.9713	3	0
3/7 votes	92.98%	95.24%	0.9260	2	6
2/7 votes	90.35%	95.24%	0.8994	2	9

Table 3. Effect of the malignant vote threshold, K-NN with k=7.

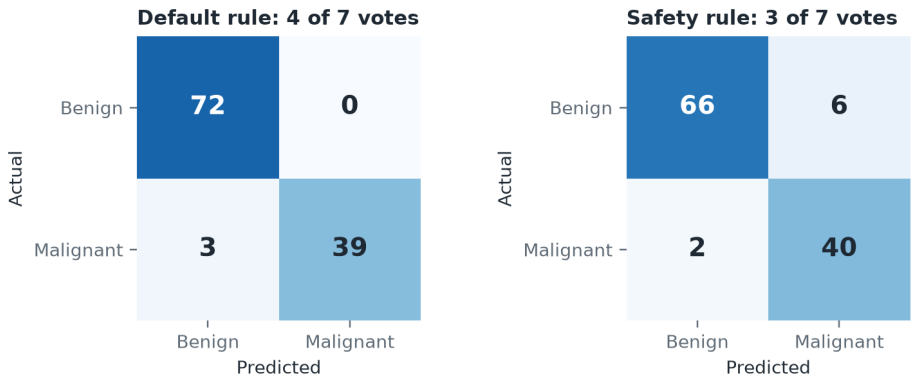


Figure 3. Confusion matrices for the default and safety rules.

Test patient #35 is a useful example. The tumour is malignant and receives 3 of 7 malignant votes, so the default rule misses it and the 3-of-7 rule catches it. The change does not remove all risk: two cancers with only one malignant vote each remain undetected, and lowering the threshold to 2 of 7 catches neither while adding more false alarms. Because this threshold was examined on the test split, the result is illustrative. A deployed threshold would need selecting on separate validation data, with clinicians weighing the cost of each error.

10. Conclusions and recommendations

- **Recommended model:** K-NN with standardised features and k=7, with an expected accuracy of 96.55% ± 1.61. It is simple, fast and explainable, and its overfitting risk is low.
- **Where missed cancers matter most:** consider the 3-of-7 rule, or the Soft-KNN ensemble, which had the fewest missed cancers (1 of 42) with 4 false alarms on this split.
- **Best next steps:** nested cross-validation for threshold selection, and external validation on data from another hospital, since one benchmark cannot show how the model generalises.

Limitations. The dataset is small and comes from a single source; test metrics carry wide margins, and the test split was consulted for several comparisons, so its figures may be slightly optimistic. The repeated cross-validation figures are the fairest estimate.

References

W. H. Wolberg, W. N. Street, O. L. Mangasarian. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository, 1995.

T. Cover, P. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 1967.

J. Goldberger, S. Roweis, G. Hinton, R. Salakhutdinov. Neighbourhood components analysis. *NeurIPS*, 2004.

I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

D. Hendrycks, K. Gimpel. Gaussian error linear units (GELUs). arXiv:1606.08415, 2016.

F. Pedregosa et al. Scikit-learn: machine learning in Python. *JMLR*, 12, 2011.

Appendix A. Reproducibility and reconciliation

Every figure in this report is generated by `reports/analysis.py` and stored in `reports/results_verified.json`, using Python 3.12.8, scikit-learn 1.9.1 and PyTorch 2.15.0+cpu on CPU. Soft-KNN values may vary slightly across library versions, and timings across machines.

To reproduce, run from the repository root:

```
pip install -r requirements.txt
python reports/analysis.py
python reports/build_report.py
```

Recomputing the team's earlier paper confirmed the core K-NN, PCA, threshold and ensemble results. The items below differ, and this report's values replace them.

Earlier claim	Verified result	Note
Convex fusion (E15): 97.37% accuracy	Not verified	No implementation is in the repository, and the results ledger records 94.74%. Excluded from this report.
97.37% accuracy with 97.6% recall	No single model	k=7 gives 97.37% with 92.86% recall; the ensemble gives 95.61% with 97.62% recall.
Zero-miss clinical recall	2 cancers missed	The 3-of-7 rule reaches 95.24% recall.
Soft-KNN AdamW: 96.49%	95.53% ± 0.92	96.49% is the best seed of 10, not the typical result.
SGD without momentum: 91.23%	94.04% ± 1.75	91.23% is the worst seed of 10.
Vectorisation 122× faster	About 22× here	Timing depends on the machine. The 1974.8 ms figure listed for Euclidean distance matches the Python-loop benchmark, not the library K-NN used for the other metrics, so those latencies are not comparable.
k=25 test accuracy: 94.74%	96.49%	Recomputed with the same split and scaling.
Gaussian jitter (std 0.3): 88.60%	96.23% ± 0.79	Jitter added as extra training copies over 10 noise seeds; the earlier setup is not in the repository.

Table 4. Differences between the earlier paper and this report.